现代数据库系统

数据库应用实践

彭煜玮
ywpeng@whu.edu.cn
珞珈图腾数据库实验室
2025.9.1

目录

CONTENTS

01 服务器端开发

02 数据库API

03 应用设计实践

PART 01

服务器端开发

PL/SQL

- Procedural Language：过程语言

- PL/SQL：过程化
  - 变量、赋值等
  - IF、ELSE
  - WHILE
  - RETURN

- 在SQL中增加控制语句使PL/SQL真正变成了一种“编程语言”

openGauss与PostgreSQL中的过程语言

- PL/pgSQL
- PL/Python
- PL/Tcl、PL/Perl、PL/Java、PL/Ruby ……

用户定义类型UDT：User Defined Type

- 在数据库中虽然定义了各种基本的数据类型，但大量的应用往往需要更多特殊的数据类型，因此用户自定义数据类型显得尤为重要。

```
CREATE TYPE AddressType AS TABLE
(
    Street NVARCHAR(50),
    City NVARCHAR(50),
    State NVARCHAR(50),
    ZipCode NVARCHAR(10)
);
```

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name NVARCHAR(100),
    Address AddressType
);
```

- 与域（Domain）的区别：域只能基于基本数据类型增加约束，UDT可以产生全新的数据类型

openGauss与PostgreSQL中的UDT

- 组合类型 CREATE TYPE compfoo AS (f1 int, f2 text);
- 枚举类型 CREATE TYPE bug_status AS ENUM ('new', 'open', 'closed');
- 范围 CREATE TYPE float8_range AS RANGE (subtype = float8, subtype_diff = float8mi);
- 全新类型

```
CREATE TYPE box;
CREATE FUNCTION my_box_in_function(cstring) RETURNS box AS ... ;
CREATE FUNCTION my_box_out_function(box) RETURNS cstring AS ... ;
CREATE TYPE box (
    INTERNALLENGTH = 16,
    INPUT = my_box_in_function,
    OUTPUT = my_box_out_function
);
CREATE TABLE myboxes (id integer, description box);
```

用户定义函数UDF：User Define Function

- 用户定义**函数**是数据库系统中非常强大的工具。它们允许你创建自定义的函数来执行重复的任务，从而提高代码的可重用性和维护性。

- UDF是一种数据库对象，本身也是一种存储在数据库中的数据

- UDF分类（根据返回值）：
  - 标量函数（Scalar Function）
  - 表值函数（Table-Valued Function）

- 其他：
  - 避免深度嵌套，维护困难
  - 业界的两极化态度

openGauss与PostgreSQL中的UDF

- CREATE FUNCTION语句

- 可用多种过程语言或外部动态库

```
CREATE OR REPLACE FUNCTION format_phone_number(
    raw_phone TEXT
) RETURNS TEXT AS $$
BEGIN
    -- 移除所有非数字字符
    raw_phone := REGEXP_REPLACE(raw_phone, '[^0-9]', '', 'g');

    -- 格式化为标准手机号格式
    IF LENGTH(raw_phone) = 11 THEN
        RETURN SUBSTRING(raw_phone FROM 1 FOR 3) || '-' ||
               SUBSTRING(raw_phone FROM 4 FOR 4) || '-' ||
               SUBSTRING(raw_phone FROM 8 FOR 4);
    ELSE
        RETURN raw_phone;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

存储过程（Stored Procedure）

- 存储过程 v.s. UDF
  - 最大区别：存储过程没有返回值
  - UDF用SELECT调用或者在SELECT中内联执行
  - 存储过程用专用语句调用：CALL my_procedure();

- 存储过程大概等价于UDP（User Defined Procedure）

openGauss与PostgreSQL中的存储过程

- CREATE PROCEDURE语句
- 其他几乎和UDF相同，但RETURN语句无参数

```
CREATE PROCEDURE insert_data(a integer, b integer)
LANGUAGE SQL
AS $$
INSERT INTO tbl VALUES (a);
INSERT INTO tbl VALUES (b);
$$;
```

```
CALL insert_data(1, 2);
```

触发器（Trigger）

触发器：是靠事件驱动的特殊过程。

- 触发器与存储过程的区别：

  存储过程通过其他程序的调用来运行，或直接启动运行，可传递参数；而触发器的启动运行必须由一个发生的事件来激发，它是自动隐式运行的，触发器不能接收参数，也不能被调用。

- 触发器的类型：
  - DML触发器：它是在执行INSERT、UPDATE或DELETE语句时被激发执行的，它只能定义在表上。
  - 替代触发器：是被激发用以代替执行DML语句，它可以定义在表或视图上。
  - 系统触发器：它是在执行CREATE、ALTER或DROP语句；执行数据库的启动或关闭、用户的登录或退出语句时被激发执行的。

触发器的主要用途

- 审计：可记录登录数据库的用户。将更新数据的用户和时间日期等信息记录在审计表中。

- 实现复杂的业务规则：如将被删除的用户保存在用户历史信息中；更新股票表只能在上市交易时间内。

- 可进行系统事件的处理：在表中内容发生变更时，自动通知其他程序采取相应的处理。

- 增强表的完整性约束：如更新职工的工资时更新后的工资不能低于更新前的工资。

触发器的结构

- 触发器的组成：
  - 触发事件
  - 触发条件
  - 触发器动作

- 触发器的一般模型：

```
CREATE TRIGGER <触发器名>
   {BEFORE|AFTER|INSTEAD OF} <触发事件>     ON <表名>|<视图名>
     [REFERENCING [OLD AS <旧元组变量>]
                  [NEW AS <新元组变量>] ]
                  [OLD  TABLE AS <旧表名>] ]
                   [NEW TABLE AS <新表名>] ]
    [FOR EACH {ROW|STATEMENT}]
    [WHEN <触发条件>]
    <SQL语句>
```

前触发器

- BEFORE 触发器 (前触发器)

在触发事件之前执行。数据还未写数据库，通过检测WHEN子句中定义的前提条件，接受或终止触发事件。

- 例：有关系 TS(sno,cno,semester,grade) 和 CL(cno,semester,limit)

创建一个触发器，它监控插入到TS中的元组，以限制课程注册的人数。

```
CREATE TRIGGER CountCheck
  BEFORE INSERT ON TS
  REFERENCING NEW AS N
  FOR EACH ROW
  WHEN ((SELECT COUNT(sno) FROM TS
            WHERE TS.cno=N.cno AND TS.semester=N.semester)
     >=(SELECT limit FROM CL
            WHERE CL.cno=N.cno AND CL.semester=N.semester))
   ROLLBACK
```

后触发器

- AFTER 触发器 (后触发器):在触发事件已经改变数据库之后执行。

[例] 规定修改SC表的grade值时，修改后的值不能低于修改前的值。

```
CREATE TRIGGER Trig_grade
AFTER UPDATE OF grade
ON SC
  REFERENCING OLD AS oldg NEW AS newg
  FOR EACH ROW
WHEN (oldg.grade>newg.grade)
  UPDATE SC
  SET grade=oldg.grade
  WHERE cno=oldg.cno AND sno=oldg.sno;
```

触发事件

触发条件

触发器动作

替代触发器

- INSTEAD OF 触发器 (替换触发器)

  INSTEAD OF触发器最普遍的应用是对视图的维护。视图上的更新事件可以通过触发器监控，但替换更新的是视图的基础表。

- 例：有关系 EMP(eno,ename,esex,salary,dno) 和 DEPT(dno,dname)

  有下面的视图：

```
CREATE VIEW WorksIn(eno,dname)
    AS
        SELECT (EMP.eno,DEPT.dname)
        FROM EMP,DEPT
        WHERE EMP.dno=DEPT.dno
```

替代触发器

- 假设在视图上执行下面的操作：

```
DELETE FROM WorksIn
 WHERE eno='11111111'
```

该操作被转换为该视图基础表EMP和DEPT上相应的操作。但是可能会有多种转换：可以删除eno为11111111的雇员所在的部门；可以删除eno为11111111的雇员；可以将雇员元组中的dno字段置为NULL。

- 无法自动决定这三种可能性中哪种是正确的。可以设计INSTEAD OF触发器来指定合适的动作。

如：

```
CREATE TRIGGER WorksInTrigi
    INSTEAD OF DELETE ON WorksIn
    REFERENCING OLD AS O
    FOR EACH ROW
        UPDATE EMP
        SET dno=NULL
        WHERE eno=O.eno
```

PART 02

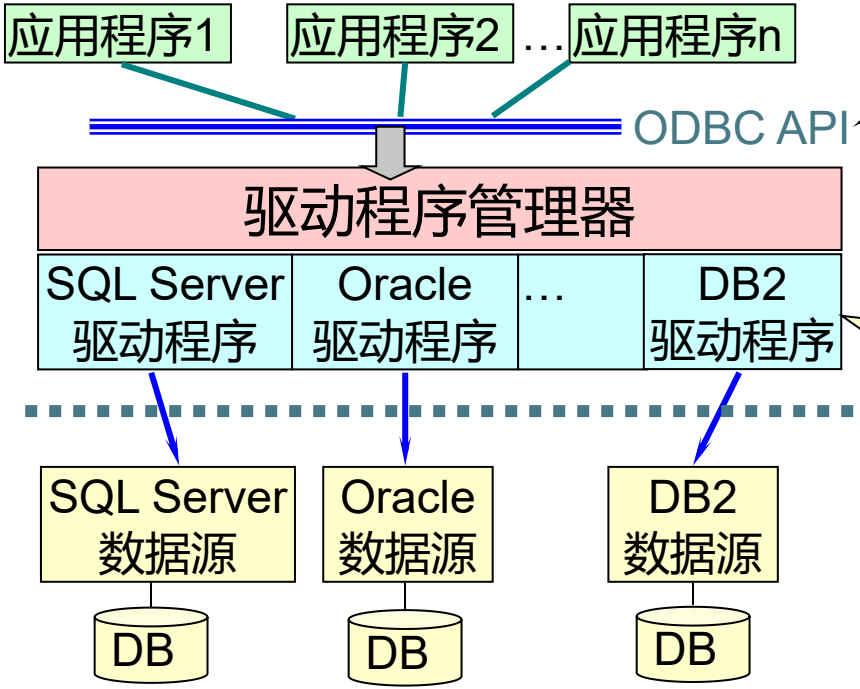
数据库API

典型的API

- **指为支持DB应用开发而提供的各种标准应用程序编程接口。**
- 1.典型的数据库应用接口
- (1) ODBC（开放式数据库互联）提供了一组对数据库访问的标准应用程序编程接口,使用户通过SQL可访问不同的关系数据库。
- (2) OLE (对象连接与嵌入数据库) 建立于 ODBC 之上，并将此技术扩展为提供更高级数据访问接口的组件结构。
- (3) JDBC(Java数据库连接)是基于Java语言的数据访问接口，具有平台无关性。适合于Internet上异构环境的数据库应用。
- (4) ADO.NET 是在微软.NET编程环境中优先使用的数据访问接口。提供了平台互用性和可伸缩的数据访问，功能强、易用、高效。

开放式接口--ODBC

(1)ODBC应用系统的体系结构：

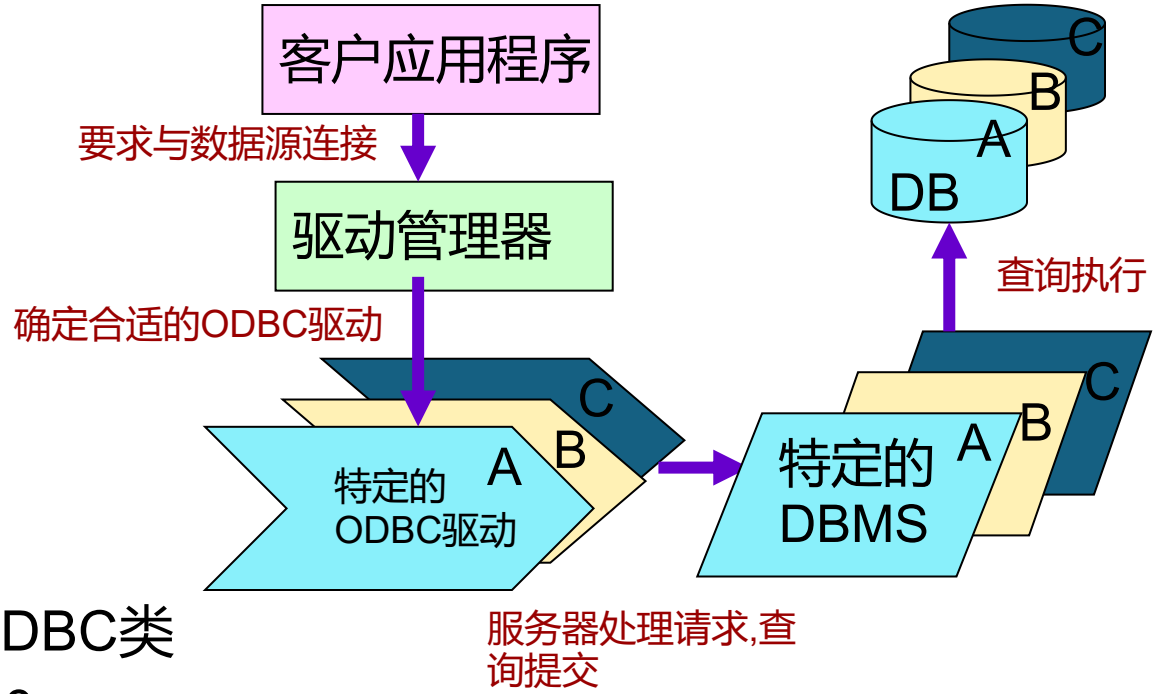


ODBC应用程序层接口:供DB应用程序开发人员使用。

由各个DB厂商提供。它提供对具体DB操作接口，是应用程序对各数据源发出操作的实际执行者

开放式接口--ODBC

(2)连接过程：



- 常用:
- MFC的ODBC类
- ODBC 3.0

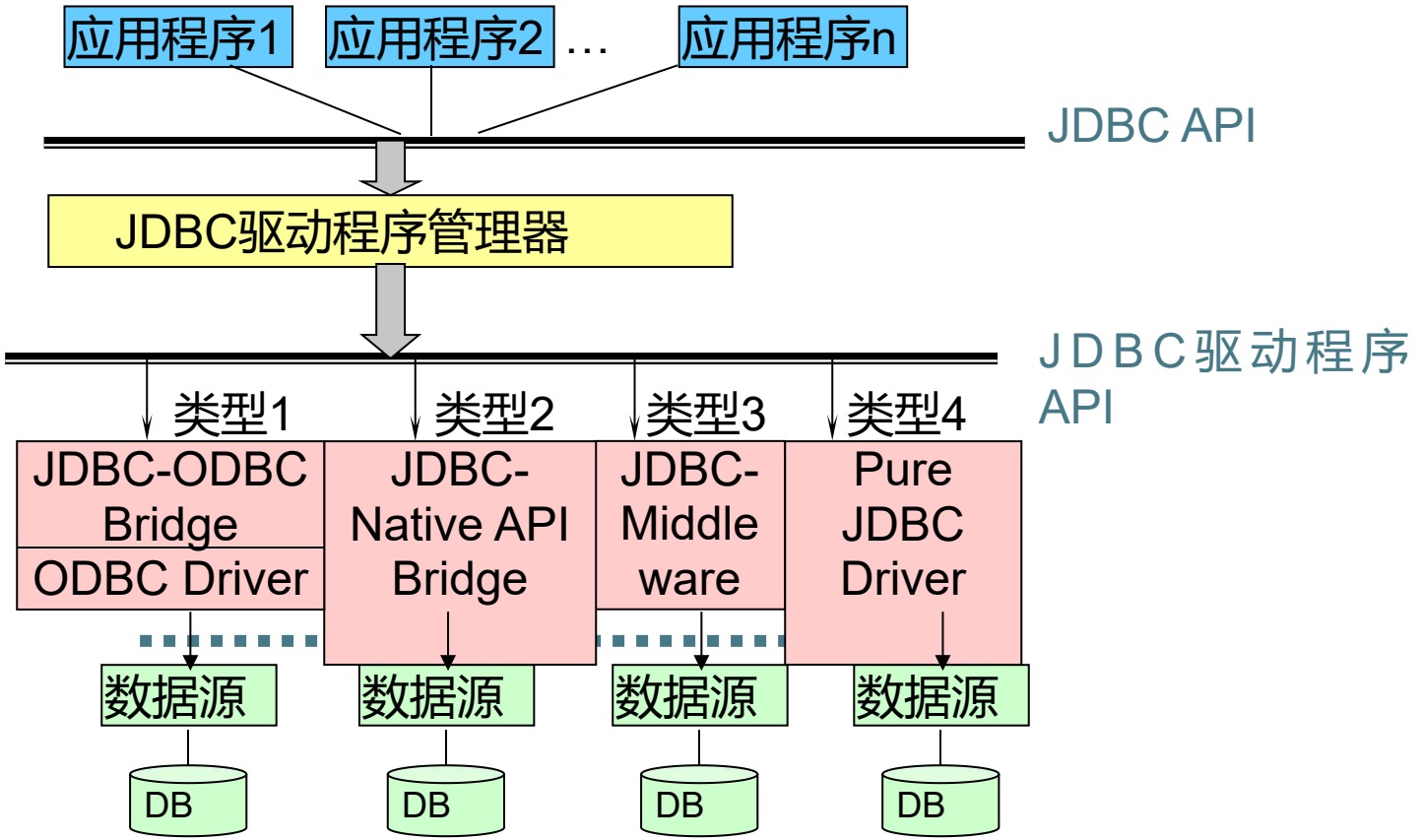
JDBC

- JDBC: Java Data Base Connectivity

- JDBC用于从Java程序调用数据库功能，是一种标准的API集合

JDBC

- JDBC类似于ODBC的方式，屏蔽了数据库产品之间的接口差异



JDBC驱动类型

- 类型1：JDBC-ODBC桥，将ODBC作为一个数据库驱动注册到JDBC中
- 类型2：JDBC-Native API桥，和类型1一样，为数据库在本地安装驱动，由JDBC调用
- 类型3：JDBC-中间件，为DBMS服务器安装中间件，由中间件来完成转换
- **类型4：纯JDBC驱动**

JDBC编程要点

①程序首部导入java.sql包

②使用Class.forName()方法加载JDBC驱动

③定义连接对象，连接到数据库

④创建操作语句对象

⑤使用SQL语句操作数据库

⑥使用close方法释放连接

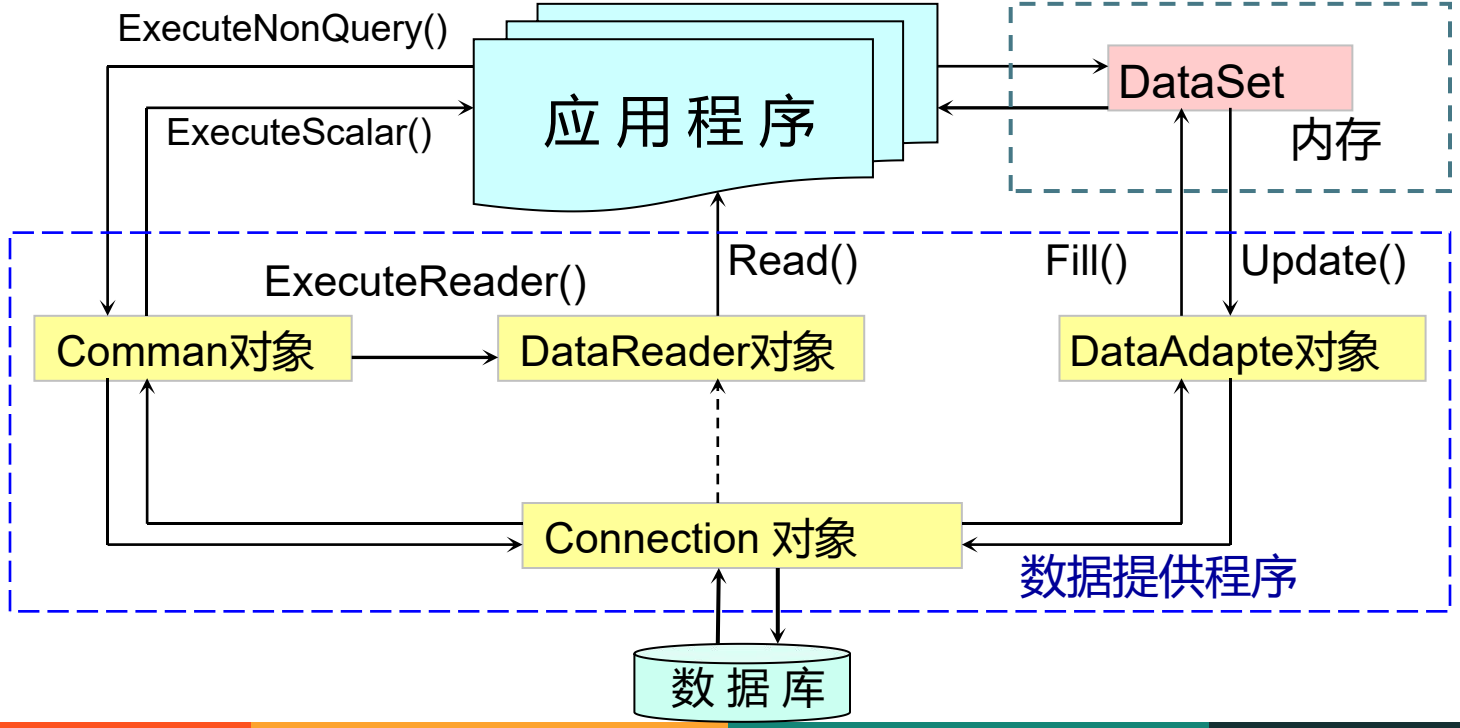
ADO.NET

- 是在微软.NET编程环境中优先使用的数据访问接口。
- 是一组用于和数据源进行交互的面向对象类库，允许和不同类型的数据源交互，功能强、易用、高效。
- 该类库中的类提供了众多对象，分别完成与数据库的连接、查询、插、删和更新记录等操作。
- **1. ADO.NET查询数据库的步骤:**

➢ 1)创建一个数据库链路;
➢ 2)请求一个记录集合;
➢ 3)把记录集合暂存到数据集(可放多个数据集);
➢ 4)关闭数据库链路;
➢ 5)在数据集上查询。

ADO.NET对象模型

- 数据提供程序主要充当DataSet及数据源之间的桥梁。



PART 03

应用设计实践

课间实验安排

- 时间：
  - 四次课12学时：第10、11周的周一、周四的3-5节
- 地点：
  - 因学院机房装修，课间实验无法在机房完成
  - 在平常上课地点进行：三区2教108
- 其他：
  - 请带上笔记本
  - 单人或结对（2人一组）完成
  - 可提前开始准备

实验内容：“寻根溯源”族谱管理系统设计与实现

一、 业务场景描述

系统中管理多个族谱，每个族谱对应于一个家族，族谱的基本信息包括：谱名、姓氏、修谱时间、创建用户。系统支持多用户，每个用户都可以创建自己的族谱，也可以邀请其他用户一起为自己创建的族谱增加成员。族谱中需记录家族成员的基本信息（姓名、性别、生卒年、生平简介）、成员间的血缘关系（父子/父女、母子/母女）以及婚姻关系。

核心挑战：

1. 成员之间的层级关系深度不确定。
2. 存在同名同姓但不同辈分的情况。
3. 需要快速查询某个成员的所有直系后代或追溯祖先。
4. 需要快速查询两个人名之间是否存在亲缘关系以及两者之间的亲缘链路。

实验内容：“寻根溯源”族谱管理系统设计与实现

二、 任务要求

1. 简单应用界面 (Application Development)

功能实现： 使用 Python (Flask/Django) 或 .NET (WinForm/WPF/WebAPI) 或其他开发工具实现一个基础图形化界面：

- 用户登录：用户只能看到和操作自己创建的或者受邀的族谱
- 用户管理：用户采用注册制
- Dashboard： 显示家族总人数、男女比例。
- 族谱、成员管理：实现增删改查、族谱能邀请他人参与创作、成员查找能支持人名的模糊查找。
- 树形预览： 以层级结构（或缩进列表）展示某分支的人物关系树状图。
- 人物祖先查询：输入一个人物的ID，以树状图显示出其父辈以上的祖先。
- 人物亲缘关系查询：输入两个人的ID，查询两者之间是否有亲缘关系连成的通路，如有请展示出来。

实验内容：“寻根溯源”族谱管理系统设计与实现

二、 任务要求

2. 数据库建模与规范化设计 (Conceptual & Logical Design)

- ER图绘制： 识别实体（成员、家庭等）及其属性，标明联系类型（1:1, 1:N, M:N）。
- 关系模式转换： 将ER图转换为关系表，并说明其达到的范式级别（建议至少满足 3NF 或 BCNF）。
- 约束设计： 定义主键、外键，并编写 CHECK 约束（如：父亲的出生年份必须早于子女）。

实验内容：“寻根溯源”族谱管理系统设计与实现

二、 任务要求

3. 数据生成与导入导出 (Data Engineering)

- 模拟数据生成： 编写 Python 脚本或 SQL 存储过程，生成至少10个族谱，其中至少有一个族谱拥有50,000以上的成员，整个系统中不少于100,000 条模拟成员数据。要求每一个族谱内的成员至少与另一个成员有亲缘关系，单个族谱内模拟出至少 30 代人的传承关系。

- 导入导出： 演示如何使用 LOAD DATA（MySQL）或 COPY（PostgreSQL）将 CSV 数据批量导入数据库，并导出某分支的备份文件。

实验内容：“寻根溯源”族谱管理系统设计与实现

二、 任务要求

4. SQL 核心功能编写 (DML & Query)

编写满足以下需求的 SQL 语句（每个需求用单个SQL实现）：

- 基本查询： 给定一个成员 ID，查询其配偶及所有子女。
- 递归查询（核心）： 编写一个递归公用表表达式（Recursive CTE），输入成员 A 的 ID，输出其向上追溯的所有历代祖先。
- 统计分析： 统计某个家族中平均寿命最长的一代人（辈分）。
- 查询所有年龄超过 50 岁（假设当前日期减去出生日期）、且没有配偶的男性成员。
- 找出家族中“出生年份”早于该辈分（代）平均出生年份的所有成员。

实验内容：“寻根溯源”族谱管理系统设计与实现

二、 任务要求

5. 物理优化与索引设计 (Physical Design)

- 索引策略： 针对“根据姓名模糊查询”和“根据父节点 ID 查询子节点”的需求，设计合适的索引。

- 性能对比： 记录在有无索引情况下，执行“查询某曾祖父的所有曾孙（四代查询）”的执行时间差异，并提交执行计划（EXPLAIN）分析。

实验内容：“寻根溯源”族谱管理系统设计与实现

三、验收要求：提交材料

1. 在实验报告中包括E-R图、关系模型、对关系模型是3NF的分析说明、创建的索引以及约束等的说明、数据的生成方法（手工插入或者自己编写工具）、所使用的RDBMS名称和版本说明、查询对应的SQL语句、在数据库中执行SQL语句后得到的结果截图。
2. 实验报告着重个人所做工作。
3. 如果采用自编的工具生成数据，请提交工具的源码。
4. 请使用你采用的RDBMS提供的数据导出或者数据备份工具将你的数据库导出，提交导出文件。

实验内容：“寻根溯源”族谱管理系统设计与实现

三、验收要求：演示

1. 演示图形化界面所要求的功能。

2. 回答老师提出的疑问。

实验内容：“寻根溯源”族谱管理系统设计与实现

四、参考系统

家谱树：https://www.familysearch.org/zh-Hant/chinese/

华谱网：https://www.zhonghuapu.com/

大谱师：https://myfamilybook.cn/

族谱网：https://www.zupu.cn/zp/